

# Explicación del código

Aplicación CRUD en Python para la tabla HR.JOBS



|               |                      |
|---------------|----------------------|
| Alumno        | Elio Castillo Reyes  |
| Matrícula     | 385770               |
| Actividad     | SQL y Python         |
| Base de datos | Oracle HR en FreeSQL |
| Fecha         | 27 de agosto de 2026 |

[https://github.com/wirenghost/DataBase1ElioCastillo\\_385770.git](https://github.com/wirenghost/DataBase1ElioCastillo_385770.git)

## 1. Objetivo y estructura del programa

La aplicación permite administrar los puestos almacenados en la tabla HR.JOBS de Oracle. Se ejecuta desde la terminal y muestra un menú con opciones para insertar, mostrar, actualizar y eliminar puestos.

### Archivos utilizados

| Archivo     | Responsabilidad                                                |
|-------------|----------------------------------------------------------------|
| database.py | Abre la conexión con Oracle y devuelve el objeto connection.   |
| app.py      | Contiene las funciones CRUD, el menú y el inicio del programa. |

### Operaciones CRUD

| Operación | Función        | Sentencia SQL |
|-----------|----------------|---------------|
| Create    | insertar_job   | INSERT        |
| Read      | mostrar_jobs   | SELECT        |
| Update    | actualizar_job | UPDATE        |
| Delete    | eliminar_job   | DELETE        |

### Importaciones necesarias en app.py

Al principio de app.py deben estar las siguientes importaciones. La primera permite capturar errores de Oracle y la segunda permite llamar a la función conectar() definida en database.py.

```
import oracledb
from database import conectar
```

## 2. Explicación de database.py

Este archivo se encarga únicamente de establecer la conexión con Oracle. Separar la conexión del resto del programa permite que app.py se concentre en las operaciones CRUD.

### Importaciones

```
import oracledb
import os
from dotenv import load_dotenv
```

- oracledb permite conectar Python con Oracle Database.
- load\_dotenv() carga variables de un archivo .env cuando existen.
- os está importado, aunque en la versión actual no se utiliza directamente.

### Función conectar()

La función conectar() llama primero a load\_dotenv() y después guarda en local\_dsn la dirección del servidor, el puerto 1521 y el servicio de Oracle.

```
local_dsn = "db.freesql.com:1521/26ai_un3c1"
```

Dentro del bloque try se ejecuta oracledb.connect(). El usuario y la contraseña se encuentran escritos directamente en database.py. Por seguridad, la contraseña no se reproduce en este reporte.

```
connection = oracledb.connect(
    user="USUARIO_DE_FREESQL",
    password="CONTRASEÑA_OMITIDA",
    dsn=local_dsn
)
```

### Prueba con DUAL

Después de conectarse, el programa crea un cursor y ejecuta SELECT \* FROM DUAL. DUAL es una tabla especial de Oracle que se utiliza para comprobar rápidamente que la conexión y la ejecución de consultas funcionan.

```
for result in cursor.execute("SELECT * FROM DUAL"):
    print(result)
```

Finalmente se cierra el cursor y se devuelve connection. Si ocurre un error de Oracle, el bloque except muestra el mensaje y devuelve None.

### 3. Funciones CREATE y READ

#### Función insertar\_job(conexion)

Esta función solicita los cuatro datos de un puesto: identificador, nombre, salario mínimo y salario máximo. Los salarios se convierten a int; si el usuario escribe texto, se produce ValueError y se muestra que deben ser números.

```
INSERT INTO HR.JOBS
(job_id, job_title, min_salary, max_salary)
VALUES (:1, :2, :3, :4)
```

- cursor.execute() envía la sentencia y la lista de valores a Oracle.
- Los marcadores :1, :2, :3 y :4 evitan concatenar los datos dentro del SQL.
- conexion.commit() confirma permanentemente la inserción.
- cursor.close() libera el cursor después de terminar.
- Si Oracle genera un error, conexion.rollback() cancela la transacción.

#### Función mostrar\_jobs(conexion)

Esta función crea un cursor, consulta todos los registros y los ordena por job\_id. fetchall() recupera las filas y las guarda en resultados.

```
SELECT job_id, job_title, min_salary, max_salary
FROM HR.JOBS
ORDER BY job_id
```

Si la lista está vacía se muestra No hay puestos registrados. En caso contrario, el ciclo for recorre cada tupla. job[0] representa el ID, job[1] el nombre, job[2] el salario mínimo y job[3] el salario máximo. SELECT no modifica datos, por lo que no necesita commit.

## 4. Funciones UPDATE y DELETE

### Función actualizar\_job(conexion)

La función solicita el job\_id que se desea modificar y los tres valores nuevos. El identificador se utiliza en la condición WHERE para actualizar únicamente el puesto seleccionado.

```
UPDATE HR.JOBS
SET job_title = :1,
    min_salary = :2,
    max_salary = :3
WHERE job_id = :4
```

Después de ejecutar el UPDATE se consulta cursor.rowcount. Si vale 0 significa que no se encontró el ID. Si es mayor que 0, se ejecuta commit() y se informa que el puesto fue actualizado.

### Función eliminar\_job(conexion)

Esta función solicita el identificador del puesto y ejecuta DELETE con una variable enlazada. La cláusula WHERE es indispensable para no eliminar todos los registros de la tabla.

```
DELETE FROM HR.JOBS
WHERE job_id = :1
```

También se revisa cursor.rowcount. Si no se afectó ninguna fila, el puesto no existe. Si se eliminó una fila, commit() confirma el cambio. Si el puesto está relacionado con empleados o historial, Oracle puede impedir la eliminación para proteger la integridad de la base de datos.

## 5. Menú y flujo de ejecución

### Función menu()

menu() llama a conectar() y almacena el resultado en la variable conexion. Si la conexión falla y se recibe None, la función termina con return.

```
conexion = conectar()

if conexion is None:
    return
```

El ciclo while True mantiene el menú activo. Dependiendo de la opción elegida, se llama a la función correspondiente y se pasa la misma conexión.

| Opción | Acción                                  |
|--------|-----------------------------------------|
| 1      | Llama a insertar_job(conexion).         |
| 2      | Llama a mostrar_jobs(conexion).         |
| 3      | Llama a actualizar_job(conexion).       |
| 4      | Llama a eliminar_job(conexion).         |
| 5      | Cierra la conexión y finaliza el ciclo. |

### Punto de entrada

La condición final comprueba que app.py se está ejecutando directamente. En ese caso se llama a menu(). Si el archivo fuera importado desde otro programa, el menú no se iniciaría automáticamente.

```
if __name__ == "__main__":
    menu()
```

### Secuencia general

- 1. Python ejecuta app.py.
- 2. app.py importa conectar desde database.py.
- 3. menu() solicita la conexión a Oracle.
- 4. El usuario elige una operación CRUD.
- 5. La función ejecuta SQL con un cursor.
- 6. Los cambios se confirman con commit o se cancelan con rollback.
- 7. Al elegir Salir se cierra la conexión.

## 6. Manejo de errores y observaciones

| Situación                  | Comportamiento                                                                    |
|----------------------------|-----------------------------------------------------------------------------------|
| Salario escrito con letras | ValueError muestra que los salarios deben ser números.                            |
| Error de Oracle            | Se muestra el error y se ejecuta rollback en las operaciones que modifican datos. |
| ID inexistente             | rowcount es 0 y se informa que el puesto no fue encontrado.                       |
| ID repetido                | Oracle puede producir ORA-00001 por llave primaria duplicada.                     |
| Puesto relacionado         | Oracle puede producir ORA-02292 al intentar eliminarlo.                           |
| Sin permisos sobre HR.JOBS | Puede aparecer ORA-01031 u ORA-00942.                                             |

### Observaciones sobre la versión actual

- app.py debe incluir import oracledb y from database import conectar al principio.
- El usuario de FreeSQL debe tener permisos de INSERT, SELECT, UPDATE y DELETE sobre HR.JOBS.
- load\_dotenv() se ejecuta, pero las credenciales actuales están escritas directamente en database.py.
- El módulo os está importado, pero no se utiliza en el código actual.
- La contraseña se omite en este reporte por seguridad.

### Conclusión

El programa cumple con las cuatro operaciones CRUD. database.py administra la conexión y app.py organiza la interacción con el usuario, las sentencias SQL. El uso de cursores, variables enlazadas, commit, rollback y rowcount permite trabajar de manera ordenada con la tabla HR.JOBS desde una aplicación de consola en Python.

Database.py

## BASES DE DATOS AVANZADAS

```
1  import oracledb
2  import os
3  from dotenv import load_dotenv
4
5  # CONEXION A ORACLE
6
7  def conectar():
8      load_dotenv()
9
10     local_dsn = "db.freesql.com:1521/26ai_un3c1"
11
12     try:
13         connection = oracledb.connect(
14             user="A385574_SCHEMA_SHQ8Z",
15             password="0lkGRMYUY98J3J!S2GN1ZT0TB2XCJZ",
16             dsn=local_dsn
17         )
18
19         print("Successfully connected to Oracle Database")
20
21         cursor = connection.cursor()
22
23         for result in cursor.execute("SELECT * FROM DUAL"):
24             print(result)
25
26         cursor.close()
27
28         return connection
29
30     except oracledb.Error as error:
31         print("Error al conectar a Oracle:", error)
32         return None
33
```

App.py



## BASES DE DATOS AVANZADAS

```
1 import oracledb
2 from database import conectar
3 # CREATE
4 def insertar_job(conexion):
5     try:
6         job_id = input("Ingresa el ID del puesto: ")
7         job_title = input("Ingresa el nombre del puesto: ")
8         min_salary = int(input("Ingresa el salario minimo: "))
9         max_salary = int(input("Ingresa el salario maximo: "))
10
11         cursor = conexion.cursor()
12
13         sql = """
14             INSERT INTO HR.JOBS
15             (job_id, job_title, min_salary, max_salary)
16             VALUES (:1, :2, :3, :4)
17         """
18
19         cursor.execute(
20             sql,
21             [job_id, job_title, min_salary, max_salary]
22         )
23
24         conexion.commit()
25
26         print("Puesto insertado correctamente.")
27
28         cursor.close()
29
30     except oracledb.Error as error:
31         print("Error al insertar:", error)
32         conexion.rollback()
33
34     except ValueError:
35         print("Los salarios deben ser numeros.")
36
```

## BASES DE DATOS AVANZADAS

```
37
38 # READ
39
40 def mostrar_jobs(conexion):
41     try:
42         cursor = conexion.cursor()
43
44         sql = """
45             SELECT job_id, job_title, min_salary, max_salary
46             FROM HR.JOBS
47             ORDER BY job_id
48         """
49
50         cursor.execute(sql)
51
52         resultados = cursor.fetchall()
53
54         print("LISTA DE JOBS")
55
56         if len(resultados) == 0:
57             print("No hay puestos registrados.")
58         else:
59             for job in resultados:
60                 print("-----")
61                 print("ID:", job[0])
62                 print("Puesto:", job[1])
63                 print("Salario minimo:", job[2])
64                 print("Salario maximo:", job[3])
65
66         cursor.close()
67
68     except oracledb.Error as error:
69         print("Error al consultar:", error)
70
71
72
```

## BASES DE DATOS AVANZADAS

```
72
73 # UPDATE
74
75 def actualizar_job(conexion):
76     try:
77         job_id = input("Ingresa el ID del puesto que quieres actualizar: ")
78
79         job_title = input("Nuevo nombre del puesto: ")
80         min_salary = int(input("Nuevo salario minimo: "))
81         max_salary = int(input("Nuevo salario maximo: "))
82
83         cursor = conexion.cursor()
84
85         sql = """
86             UPDATE HR.JOBS
87             SET job_title = :1,
88                 min_salary = :2,
89                 max_salary = :3
90             WHERE job_id = :4
91         """
92
93         cursor.execute(
94             sql,
95             [job_title, min_salary, max_salary, job_id]
96         )
97
98         if cursor.rowcount == 0:
99             print("No se encontro un puesto con ese ID.")
100         else:
101             conexion.commit()
102             print("Puesto actualizado correctamente.")
103
104         cursor.close()
105
106     except oracledb.Error as error:
107         print("Error al actualizar:", error)
108         conexion.rollback()
109
110     except ValueError:
111         print("Los salarios deben ser numeros.")
```

## BASES DE DATOS AVANZADAS

```
111 |         print("Los salarios deben ser numeros.")
112 |
113 |
114 |
115 | # DELETE
116 |
117 |
118 | def eliminar_job(conexion):
119 |     try:
120 |         job_id = input("Ingresa el ID del puesto que quieres eliminar: ")
121 |
122 |         cursor = conexion.cursor()
123 |
124 |         sql = """
125 |             DELETE FROM HR.JOBS
126 |             WHERE job_id = :1
127 |         """
128 |
129 |         cursor.execute(sql, [job_id])
130 |
131 |         if cursor.rowcount == 0:
132 |             print("No se encontro un puesto con ese ID.")
133 |         else:
134 |             conexion.commit()
135 |             print("Puesto eliminado correctamente.")
136 |
137 |         cursor.close()
138 |
139 |     except oracledb.Error as error:
140 |         print("Error al eliminar:", error)
141 |         conexion.rollback()
142 |
143 |
144 |
145 | # MENU
146 |
147 |
148 | def menu():
149 |     conexion = conectar()
150 |
151 |     if conexion is None:
152 |         return
```

## BASES DE DATOS AVANZADAS

```
153
154     while True:
155
156         print("CRUD")
157         print("1. Insertar puesto")
158         print("2. Mostrar puestos")
159         print("3. Actualizar puesto")
160         print("4. Eliminar puesto")
161         print("5. Salir")
162         opcion = input("Selecciona una opcion: ")
163
164         if opcion == "1":
165             insertar_job(conexion)
166
167         elif opcion == "2":
168             mostrar_jobs(conexion)
169
170         elif opcion == "3":
171             actualizar_job(conexion)
172
173         elif opcion == "4":
174             eliminar_job(conexion)
175
176         elif opcion == "5":
177             print("Programa finalizado.")
178             conexion.close()
179             break
180
181         else:
182             print("Opcion no valida.")
183
184 if __name__ == "__main__":
185     menu()
```